32-bit Arithmetic Logic Unit (ALU) using Verilog HDL

Mini Project Report:

Submitted By: Munthaj Ameena sayyed
Department: Electrical and Electronics Engineering (EEE)
College: Rajiv Gandhi University of Knowledge Technologies (RGUKT), Nuzvid
Academic Year: 2024–2028

Abstract

The Arithmetic Logic Unit (ALU) is one of the most important components of a processor. It performs arithmetic and logical operations required for data processing. This project presents the design and implementation of a 32-bit ALU using Verilog HDL. The ALU supports operations such as addition, subtraction, bitwise AND, OR, XOR, NOT, left shift, right shift, increment, and decrement. The design is simulated using Xilinx Vivado to verify the correctness of each operation. This project improves the understanding of combinational logic design, RTL coding, and digital hardware implementation.

Introduction

An Arithmetic Logic Unit (ALU) is a combinational digital circuit that performs mathematical and logical operations on binary data. It acts as the computational core of CPUs, microprocessors, and embedded systems. The ALU receives two input operands and performs the selected operation based on a control signal. A 32-bit ALU can process larger data values with better accuracy and speed than smaller ALUs, making it suitable for modern digital systems.

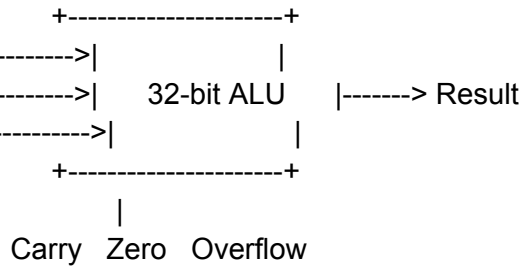
Objectives

- Design a 32-bit ALU using Verilog HDL.
- Implement arithmetic and logical operations.
- Generate Carry, Zero, and Overflow status flags.
- Simulate the design using Xilinx Vivado.
- Verify the output using a Verilog testbench.
- Understand RTL design and combinational logic concepts.

Software Requirements

Software	Purpose
Xilinx Vivado	Design and Simulation
Verilog HDL	Hardware Description Language
Visual studio code	coding
Icarus verilog	compiler
Gtkwave	waveform

Block Diagram



Input and Output Description

Inputs

A (32-bit): First input operand.

B (32-bit): Second input operand.

SEL (4-bit): Selects the operation to be performed.

Outputs

Result (32-bit): Stores the output of the selected operation.

Carry: Indicates carry generated during arithmetic operations.

Zero: Becomes HIGH when the result is zero.

Overflow: Indicates signed arithmetic overflow.

Operations Performed

Select| Operation
0000| Addition
0001| Subtraction
0010| AND
0011| OR
0100| XOR
0101| NOT
0110| Left Shift
0111| Right Shift
1000| Increment
1001| Decrement

Working Principle

The ALU continuously monitors the values of inputs A, B, and the selection signal. Whenever any input changes, the combinational logic immediately calculates the new output. A case statement is used to select the required operation based on the value of the selection signal. The result is stored in a 32-bit output register, while the Carry, Zero, and Overflow flags are updated whenever required. This design is fully combinational, so no clock signal is needed.

Verilog Design

The design is implemented using behavioral modeling in Verilog HDL.

The Verilog module includes:

- Module declaration
- Input and output ports
- Temporary 33-bit register for carry detection
- always @(*) block
- case statement
- Carry, Zero, and Overflow flag generation

Verilog Code:

```
module alu32(
    input  [31:0] A,
    input  [31:0] B,
    input  [3:0] sel,
    output reg [31:0] result,
    output reg carry,
    output reg zero,
    output reg overflow
);

reg [32:0] temp;

always @(*) begin
    carry = 0;
    overflow = 0;

    case(sel)

        4'b0000: begin              // Addition
            temp = A + B;
            result = temp[31:0];
            carry = temp[32];
            overflow = (A[31] == B[31]) &&
                       (result[31] != A[31]);
        end

        4'b0001: begin              // Subtraction
            temp = A - B;
            result = temp[31:0];
            carry = temp[32];
            overflow = (A[31] != B[31]) &&
                       (result[31] != A[31]);
        end

        4'b0010: result = A & B;    // AND

        4'b0011: result = A | B;    // OR

        4'b0100: result = A ^ B;    // XOR

        4'b0101: result = ~A;       // NOT
```

```
        4'b0110: result = A << 1;   // Left Shift

        4'b0111: result = A >> 1;   // Right Shift

        4'b1000: result = A + 1;    // Increment

        4'b1001: result = A - 1;    // Decrement

        default: result = 32'b0;

    endcase

    if(result == 32'b0)
        zero = 1;
    else
        zero = 0;

end

endmodule
```

Testbench

A separate Verilog testbench is created to verify the functionality of the ALU. Different input values are applied, and every operation is tested by changing the selection signal. The simulation waveform confirms that the ALU performs each operation correctly.

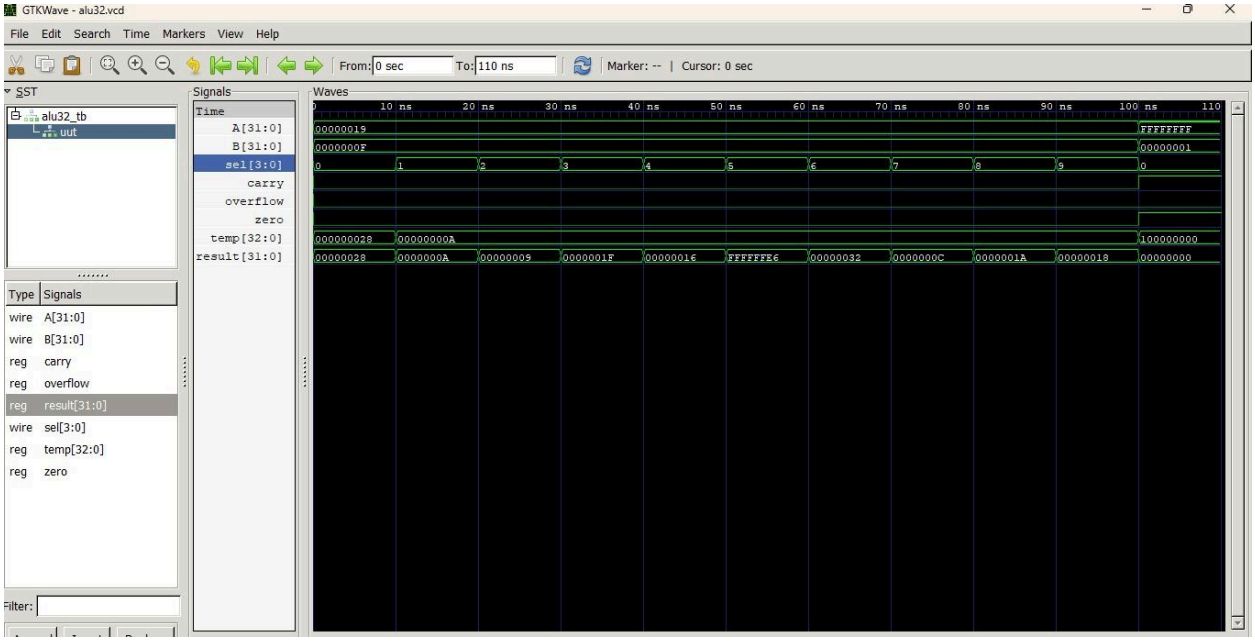
Testbench code:

```
`timescale 1ns/1ps

module alu32_tb;

reg [31:0] A;
reg [31:0] B;
reg [3:0] sel;

wire [31:0] result;
wire carry;
wire zero;
wire overflow;
```

```
alu32 uut(
    .A(A),
    .B(B),
    .sel(sel),
    .result(result),
    .carry(carry),
    .zero(zero),
    .overflow(overflow)
);

initial begin

    A = 32'd25;
    B = 32'd15;

    sel = 4'b0000; #10;   // ADD
    sel = 4'b0001; #10;   // SUB
    sel = 4'b0010; #10;   // AND
    sel = 4'b0011; #10;   // OR
    sel = 4'b0100; #10;   // XOR
    sel = 4'b0101; #10;   // NOT
    sel = 4'b0110; #10;   // Left Shift
    sel = 4'b0111; #10;   // Right Shift
    sel = 4'b1000; #10;   // Increment
    sel = 4'b1001; #10;   // Decrement

    A = 32'hFFFFFFFF;
    B = 32'd1;
    sel = 4'b0000; #10;

    $finish;

end

endmodule
```

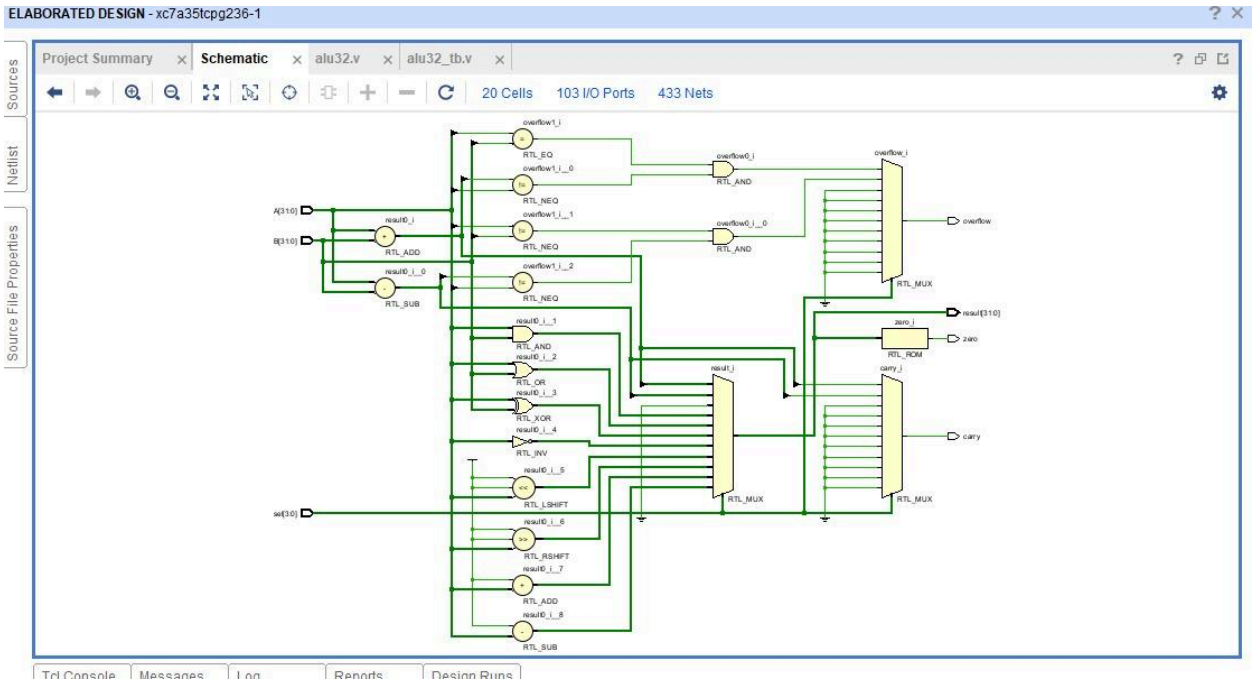
Simulation Results

The design was successfully simulated using Xilinx Vivado. The simulation verified the correct functionality of addition, subtraction, logical operations, shift operations, increment, and decrement. The Carry, Zero, and Overflow flags were generated correctly for the tested input combinations.



—

RTL schematic diagram



—

Applications

- Central Processing Units (CPUs)
- Microprocessors
- Embedded Systems
- FPGA Designs
- Digital Signal Processing (DSP)
- Arithmetic Processing Units
- Digital Controllers

Advantages

- High-speed operation
- Supports multiple arithmetic and logical functions
- Simple and modular design
- Easy to synthesize on FPGA
- Easy to modify and expand
- Useful for processor design

Conclusion

The 32-bit Arithmetic Logic Unit was successfully designed and simulated using Verilog HDL in Xilinx Vivado. The ALU correctly performs arithmetic, logical, and shift operations based on the selection input. The simulation results verified the correctness of all implemented operations. This project provided practical knowledge of RTL design, Verilog programming, combinational circuits, and digital system implementation. It also serves as a strong foundation for advanced VLSI and FPGA-based processor design projects.
